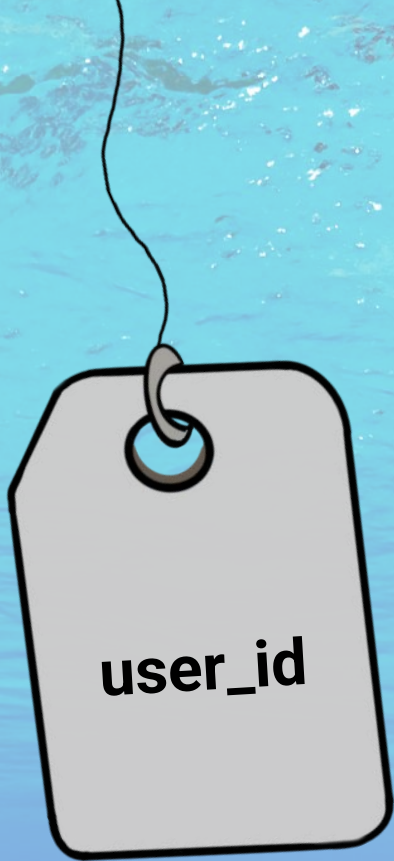
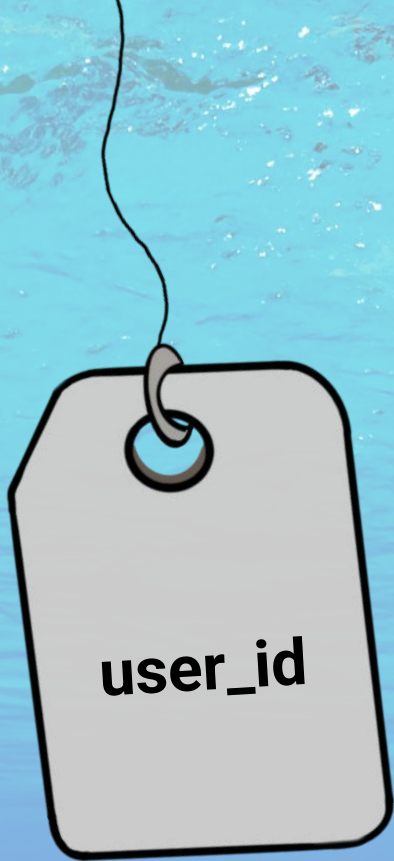
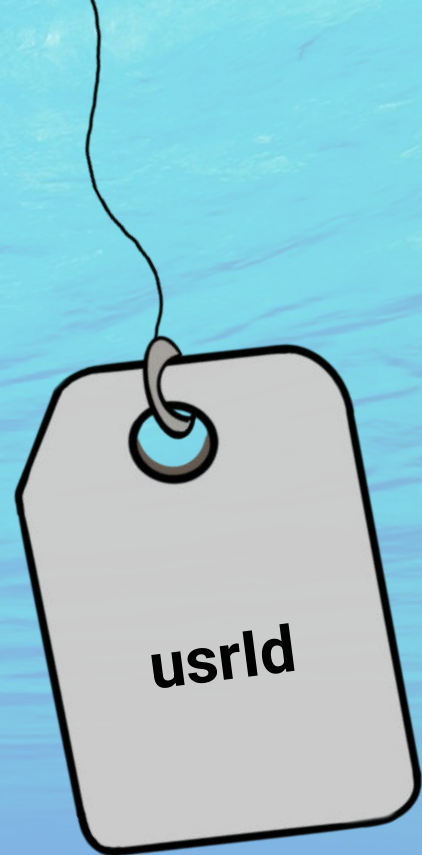
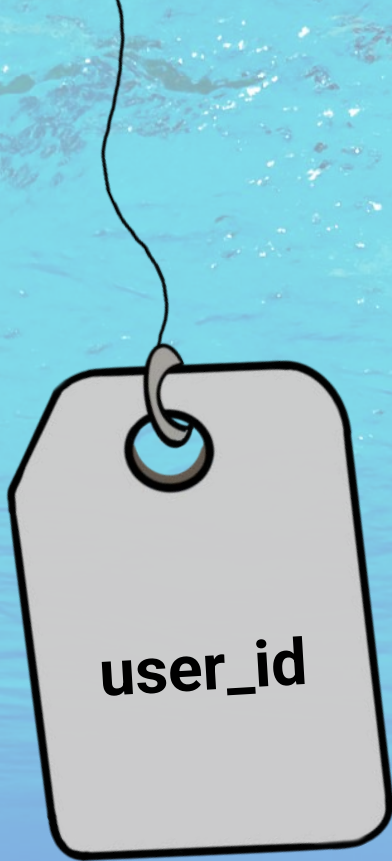


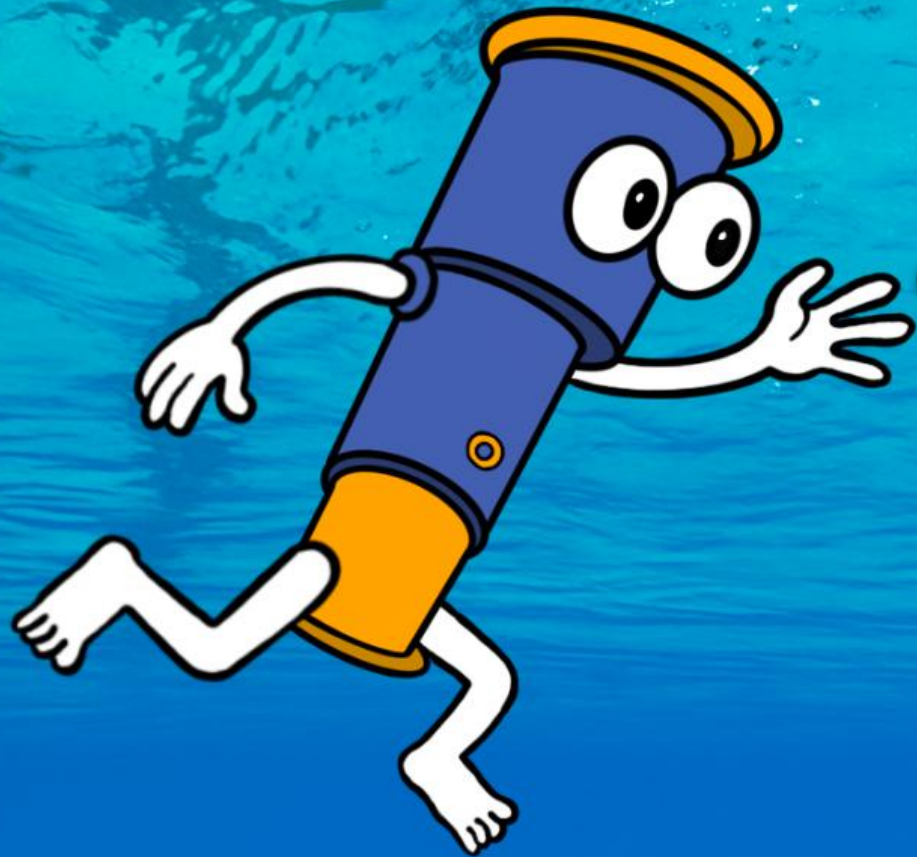


user.id

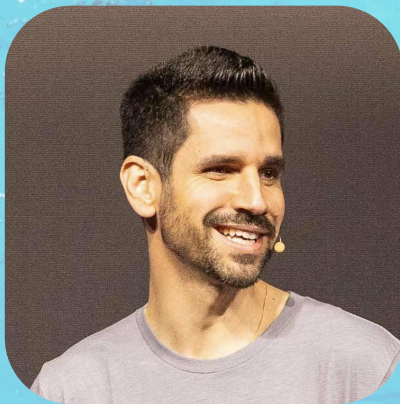






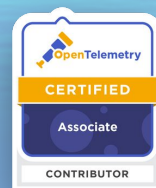


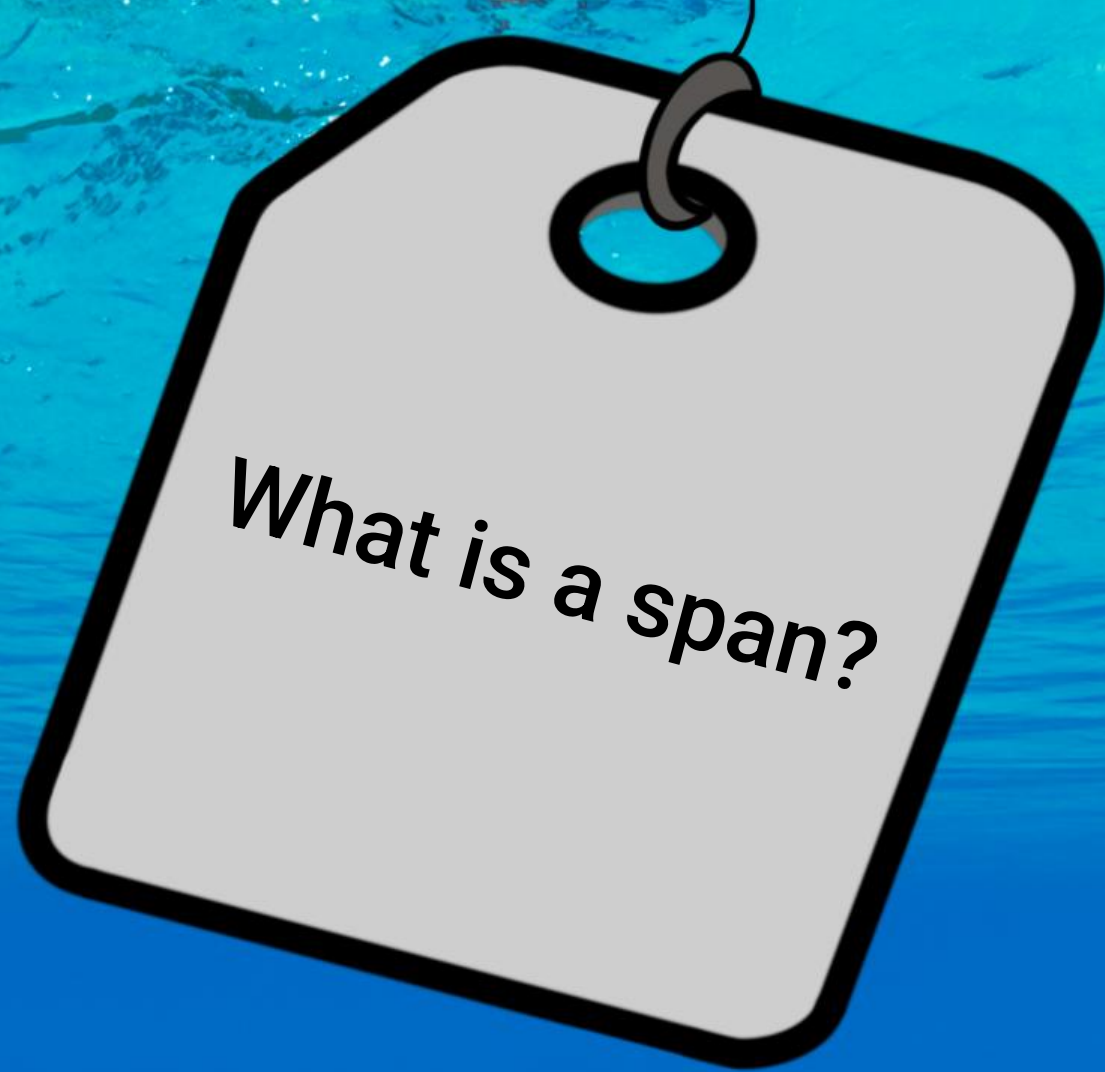
Stop the Noise, Create Better Spans



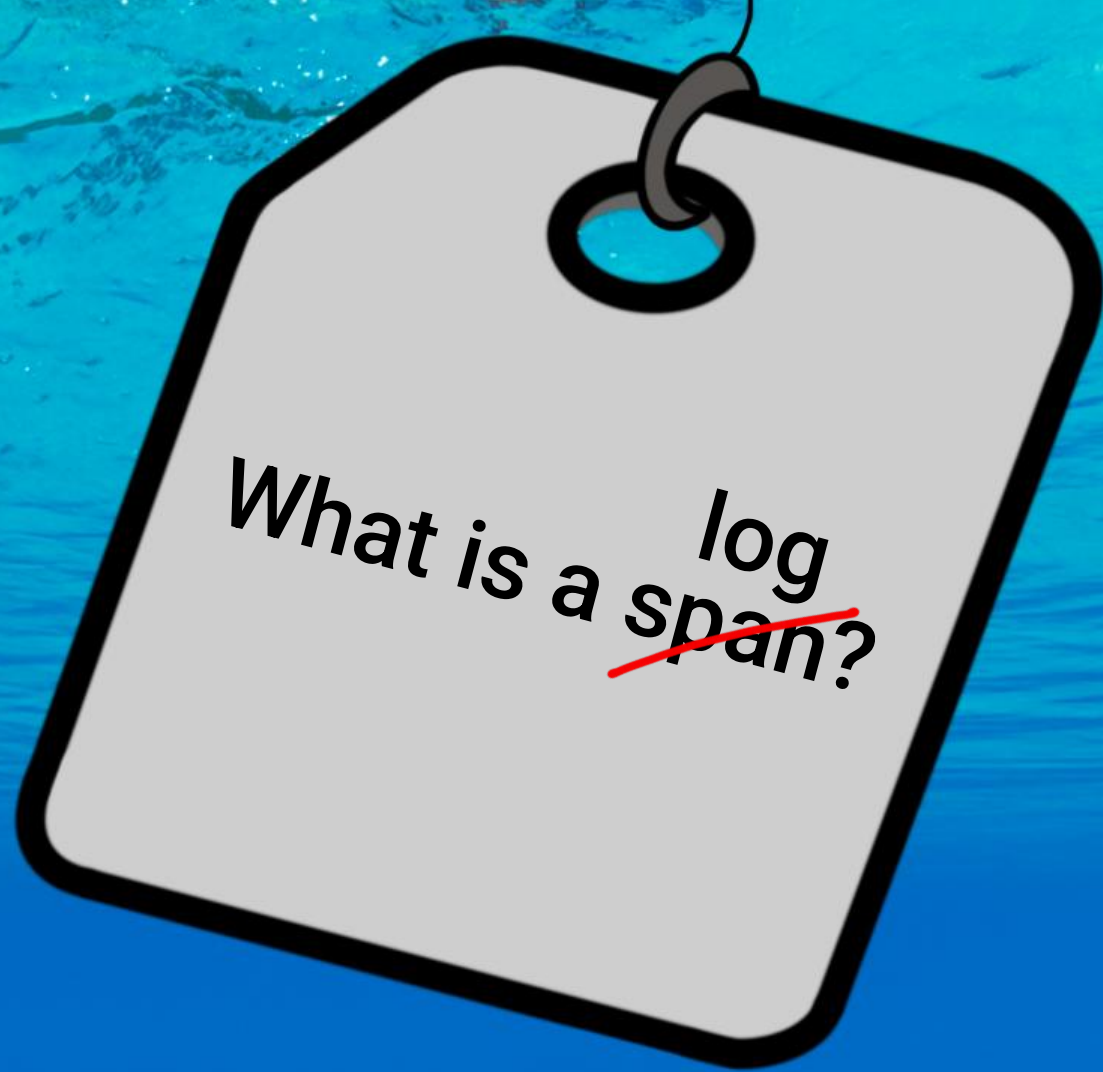
Juliano Costa

Developer Advocate





What is a span?



What is a ~~span~~ log?

What is a log?

24.09

What is a log?

24.09

13:06:30 - Listened to Smells Like Teen Spirit

What is a log?

24.09

13:06:30 - Listened to Smells Like Teen Spirit

13:11:31 - Listened to Come As You Are

What is a log?

24.09

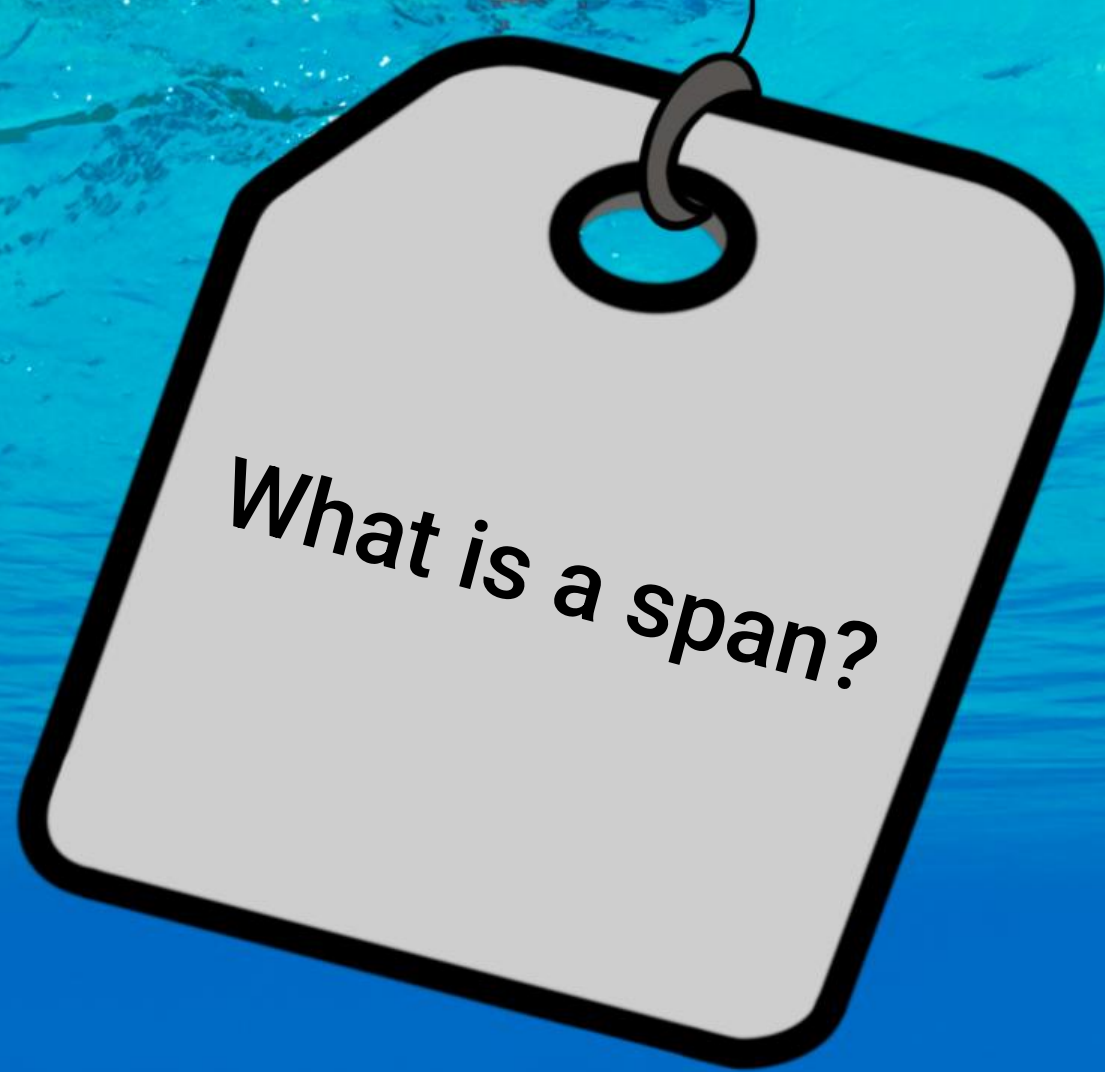
13:06:30 - Listened to Smells Like Teen Spirit

13:11:31 - Listened to Come As You Are

13:15:09 - Listened to Something In The Way

What is a log?

↓ DATE	HOST	SERVICE	STATUS	CONTENT
 Aug 27 11:00:13.000	teen-spirit	nevermind-api	critical	Oh well, whatever-nevermind. Shutting down.
 Aug 27 11:00:12.000	teen-spirit	nevermind-api	error	Drain you, data. Connection lost.
 Aug 27 11:00:11.000	teen-spirit	nevermind-api	error	All apologies. Task execution failed.
 Aug 27 11:00:09.000	teen-spirit	nevermind-api	warn	Just because you're paranoid, check those jobs.
 Aug 27 11:00:08.000	teen-spirit	nevermind-api	info	I'm so happy; today I found my task.
 Aug 27 11:00:06.000	teen-spirit	nevermind-api	warn	Something in the way... request delayed.
 Aug 27 11:00:04.000	teen-spirit	nevermind-api	info	Come as you are, as you logged in.
 Aug 27 11:00:03.000	teen-spirit	nevermind-api	debug	I swear I don't have a key.
 Aug 27 11:00:02.000	teen-spirit	nevermind-api	info	Here we are now, authenticate us.
 Aug 27 11:00:01.000	teen-spirit	nevermind-api	info	Load up on users, bring your friends.



What is a span?

What is a span?

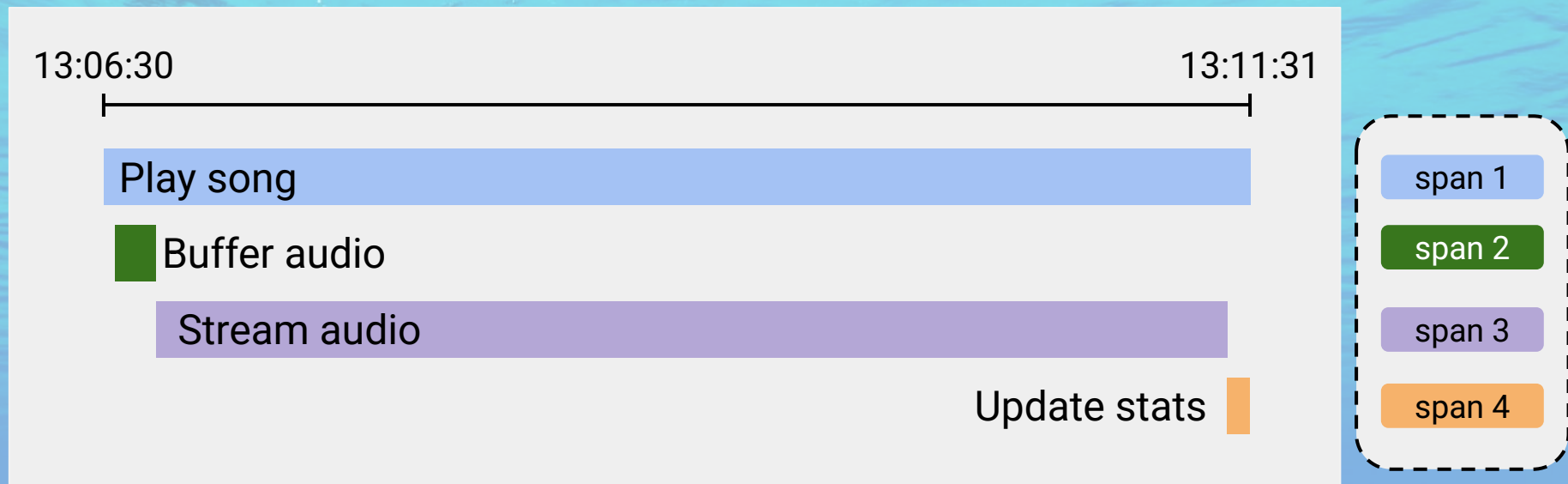
24.09

13:06:30 - Listened to Smells Like Teen Spirit

13:11:31 - Listened to Come As You Are

13:15:09 - Listened to Something In The Way

What is a span?



What is a span?

Play song

What is a span?

Play song

```
{  
  "name": "Play song",  
  "context": {  
    "trace_id": "5b8aa5a2d2c872e8321cf37308d69df2",  
    "span_id": "051581bf3cb55c13"  
  },  
  "parent_id": null,  
  "start_time": "1991-09-24T13:06:30.114201Z",  
  "end_time": "1991-09-24T13:11:31.114687Z",  
  "attributes": {  
    "media.song.name": "Smells Like Teen Spirit"  
  }  
}
```


What is a span?

Play song

```
{
  "name": "Play song",
  "context": {
    "trace_id": "5b8aa5a2d2c872e8321cf37308d69df2",
    "span_id": "051581bf3cb55c13"
  },
  "parent id": null,
  "start_time": "1991-09-24T13:06:30.114201Z",
  "end_time": "1991-09-24T13:11:31.114687Z",
  "attributes": {
    "media.song.name": "Smells Like Teen Spirit"
  }
}
```

What is a span?

Play song

```
{
  "name": "Play song",
  "context": {
    "trace_id": "5b8aa5a2d2c872e8321cf37308d69df2",
    "span_id": "051581bf3cb55c13"
  },
  "parent_id": null,
  "start_time": "1991-09-24T13:06:30.114201Z",
  "end_time": "1991-09-24T13:11:31.114687Z",
  "attributes": {
    "media.song.name": "Smells Like Teen Spirit"
  }
}
```

What is a span?

Play song

```
{
  "name": "Play song",
  "context": {
    "trace_id": "5b8aa5a2d2c872e8321cf37308d69df2",
    "span_id": "051581bf3cb55c13"
  },
  "parent_id": null,
  "start_time": "1991-09-24T13:06:30.114201Z",
  "end_time": "1991-09-24T13:11:31.114687Z",
  "attributes": {
    "media.song.name": "Smells Like Teen Spirit"
  }
}
```

What is a span?

Play song

```
"context": {  
  "trace_id": "5b8aa5a2d2c872e8321cf37308d69df2",  
  "span_id": "051581bf3cb55c13"  
},
```



Buffer audio

What is a span?

 Buffer audio

```
{  
  "name": "Buffer audio",  
  "context": {  
    "trace_id": "5b8aa5a2d2c872e8321cf37308d69df2",  
    "span_id": "9faed88b3f9ed699"  
  },  
  "parent_id": "051581bf3cb55c13",  
  "start_time": "1991-09-24T13:06:35.114201Z",  
  "end_time": "1991-09-24T13:07:00.114687Z",  
  "attributes": {  
    "media.song.name": "Smells Like Teen Spirit"  
  }  
}
```



Attributes

Span Attributes

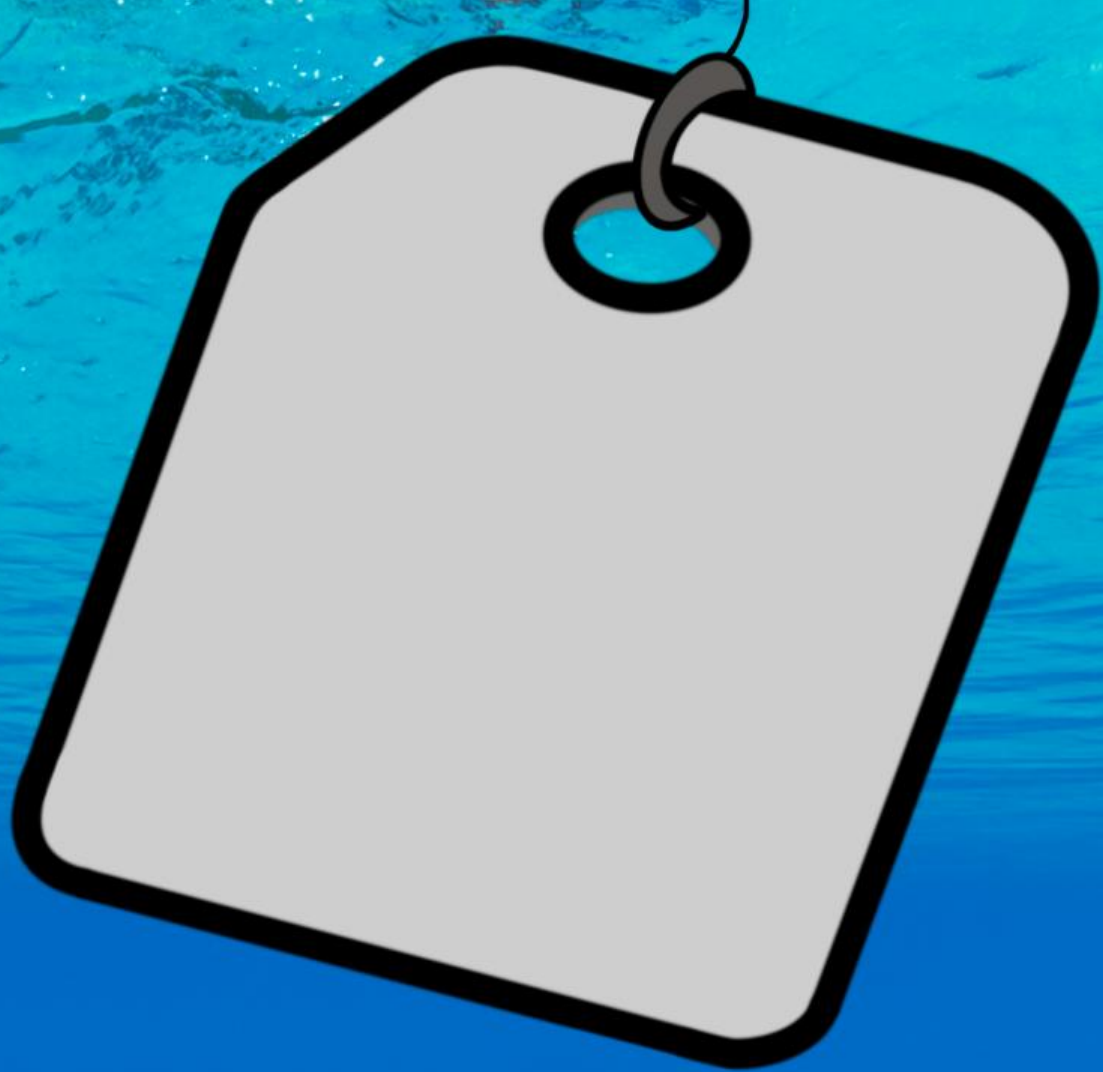
```
{
  "name": "Play song",
  "context": {
    "trace_id": "5b8aa5a2d2c872e8321cf37308d69df2",
    "span_id": "051581bf3cb55c13"
  },
  "parent_id": null,
  "start_time": "1991-09-24T13:06:30.114201Z",
  "end_time": "1991-09-24T13:11:31.114687Z",
  "attributes": {
    "media.song.name": "Smells Like Teen Spirit"
  }
}
```

Span Attributes

```
{
  "name": "Play song",
  "context": {
    "trace_id": "5b8aa5a2d2c872e8321cf37308d69df2",
    "span_id": "051581bf3cb55c13"
  },
  "parent_id": null,
  "start_time": "1991-09-24T13:06:30.114201Z",
  "end_time": "1991-09-24T13:11:31.114687Z",
  "attributes": {
    "media.song.name": "Smells Like Teen Spirit",
    "user.id": "92590",
    "user.subscription.type": "premium"
  }
}
```


Span Attributes

```
{
  "...": "...",
  "attributes": {
    "media.song.name": "Smells Like Teen Spirit",
    "user.id": "92590",
    "user.subscription.type": "premium",
    "media.data": [{
      "id": "a3a869c2-4f52-3649-b118-2b81f2a6b7f8",
      "position": 1,
      "format": "CD",
      "track": [{
        "id": "a3456745-01e6-3e30-b8a9-bbf8b54faf10",
        "number": "1",
        "title": "Smells Like Teen Spirit",
        "length": 276333
      }],
      "track-count": 17,
      "track-offset": 0
    }]
  }
}
```





Logs x Spans

Logs x Spans

```
{  
  "timestamp": "2025-08-05T16:21:12.842Z",  
  "user_id": "eddie_vedder_123",  
  "action": "read",  
  "resource": "/nirvana/unreleased_songs.pdf",  
  "status": "failure",  
  "reason": "Insufficient permissions",  
  "event_type": "audit_log",  
  "compliance_tag": "gdpr"  
}
```


Logs x Spans

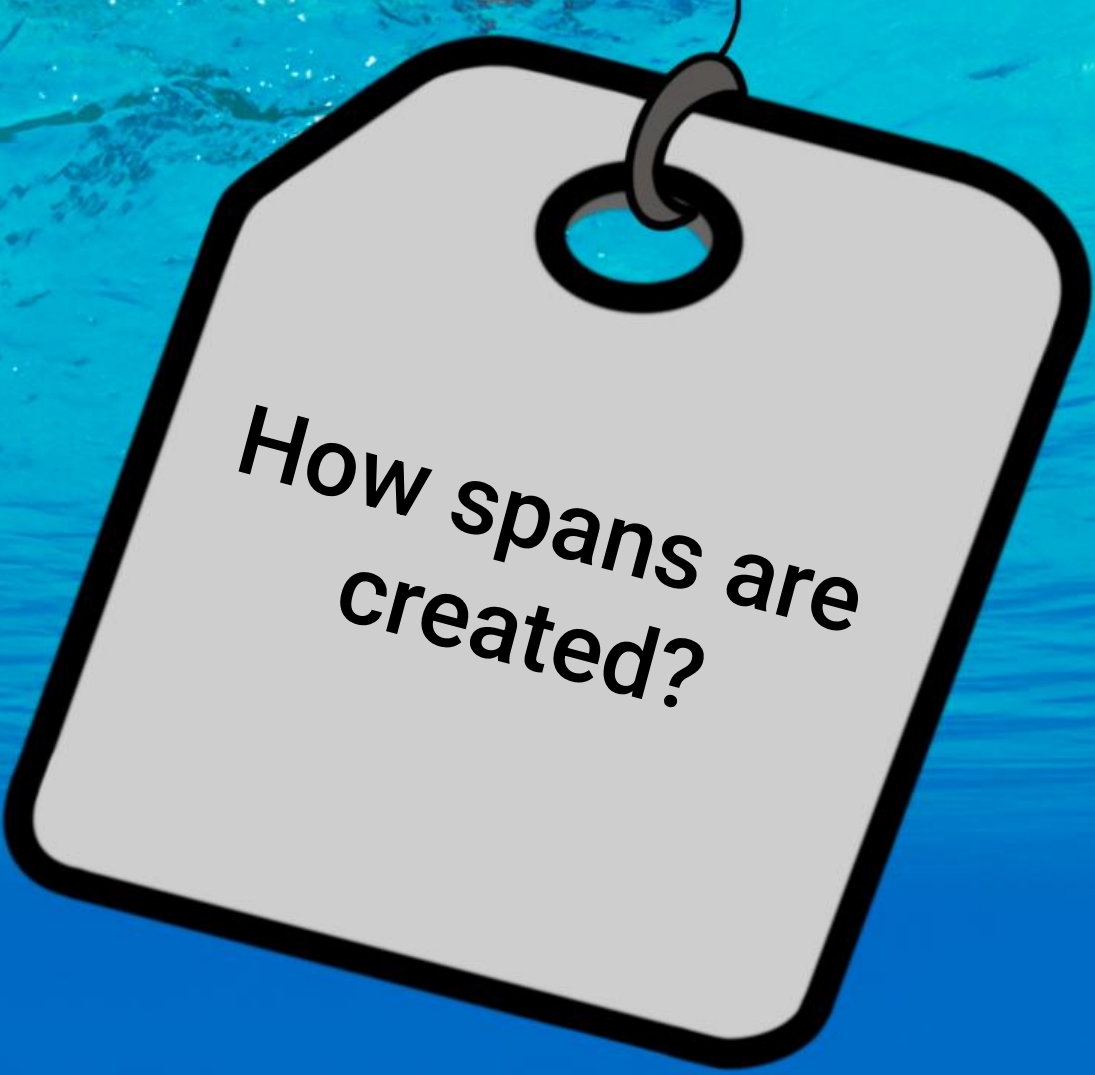
```
def get_songs(title, artist):  
    logging.info("Get song for title: %s, artist: %s", title, artist)  
  
    # Application logic to retrieve song metadata  
    # [...]  
  
    logging.info("Song retrieved for title: %s, artist: %s", title, artist)  
    return [...]
```

Logs x Spans

```
@tracer.start_as_current_span("GET /songs/{title}/{artist}", kind=trace.SpanKind.SERVER)
def get_songs(title, artist):

    # Application logic to retrieve song metadata
    # [...]

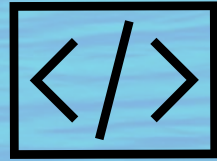
    return [...]
```



**How spans are
created?**



Auto instrumentation



Auto instrumentation - how to

```
curl -L -O https://github.com/open-telemetry/opentelemetry-java-  
instrumentation/releases/latest/download/opentelemetry-javaagent.jar
```

```
export JAVA_TOOL_OPTIONS=-javaagent:/usr/src/app/opentelemetry-javaagent.jar
```

```
java -jar myapp.jar
```

Auto instrumentation - what you get

Traces >  songs-auto > GET /songs/{title}/{artist} > trace_id 38afea11da8320de2cd1497827f44089

 15.5ms | GET 200 OK Aug 28 16:37:30.069 (3d ago)

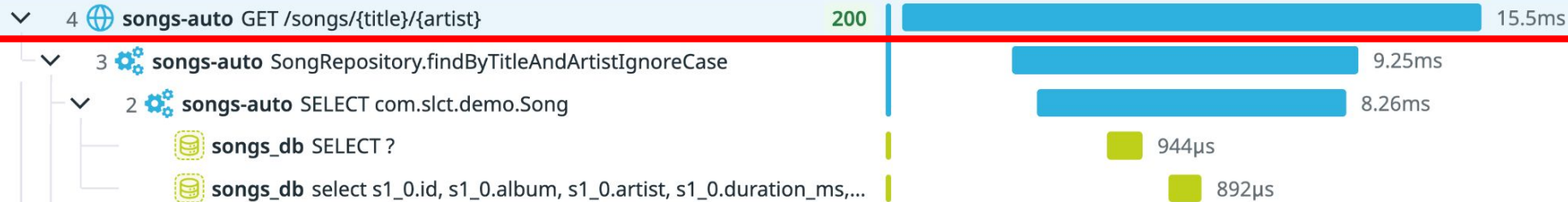
Trace: Flame Graph Waterfall Span List 5 Map

Filter spans by any attribute

Color by Service ▼



0 5 ms 10 ms 15 m



Auto instrumentation - what you get

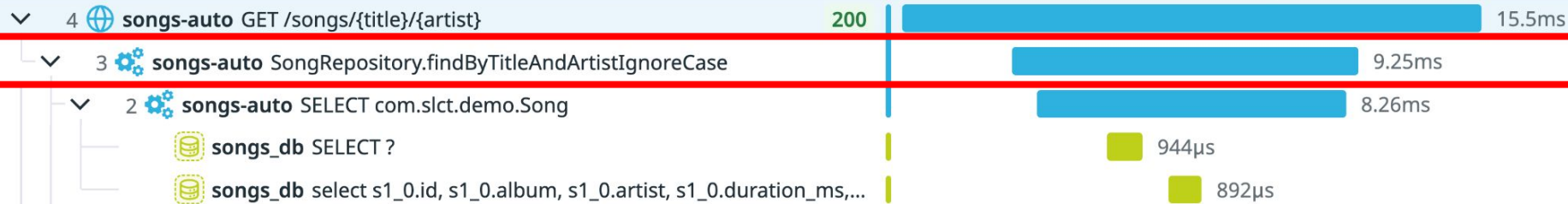
Traces >  songs-auto > GET /songs/{title}/{artist} > trace_id 38afea11da8320de2cd1497827f44089

p29  15.5ms | GET 200 OK Aug 28 16:37:30.069 (3d ago)

Trace: Flame Graph Waterfall Span List 5 Map

 Filter spans by any attribute

Color by Service ▼



Auto instrumentation - what you get

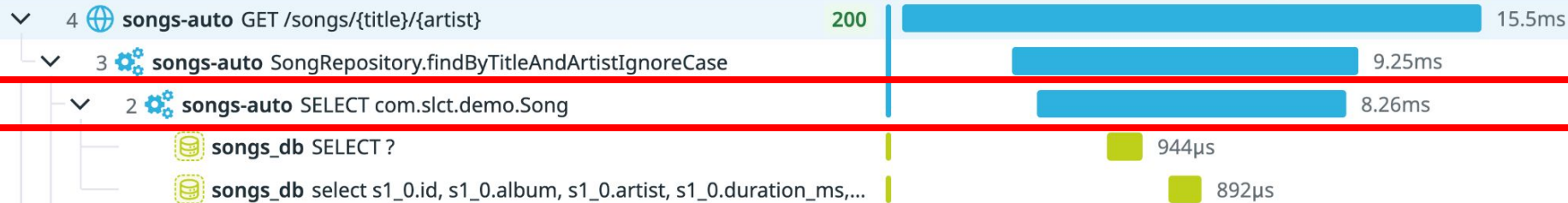
Traces >  songs-auto > GET /songs/{title}/{artist} > trace_id 38afea11da8320de2cd1497827f44089

p29  15.5ms | GET 200 OK Aug 28 16:37:30.069 (3d ago)

Trace: Flame Graph Waterfall Span List 5 Map

 Filter spans by any attribute

Color by Service 



Auto instrumentation - what you get

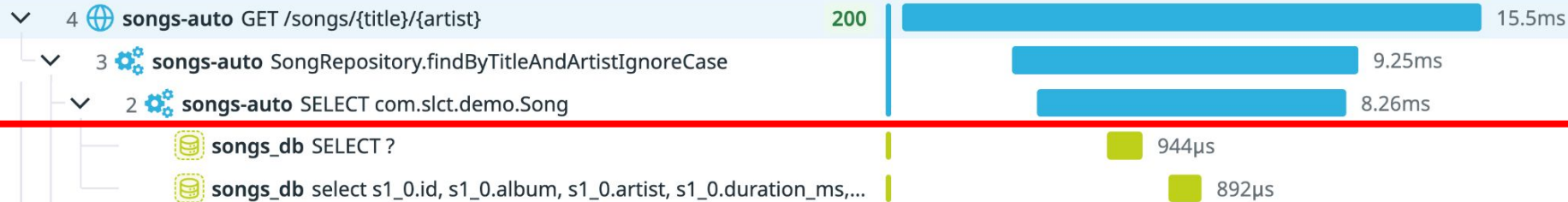
Traces >  songs-auto > GET /songs/{title}/{artist} > trace_id 38afea11da8320de2cd1497827f44089

p29  15.5ms | GET 200 OK Aug 28 16:37:30.069 (3d ago)

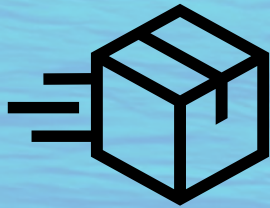
Trace: Flame Graph Waterfall Span List 5 Map

 Filter spans by any attribute

Color by Service ▼



Instrumentation libraries



Instrumentation libraries - how to

```
use actix_web::{App, HttpServer};
use opentelemetry_instrumentation_actix_web::RequestTracing;

mod handlers;
mod telemetry_conf;

#[actix_web::main]
async fn main() -> std::io::Result<()> {
    telemetry_conf::init_otel().expect("Failed to initialize OpenTelemetry");

    HttpServer::new(|| {
        App::new()
            .wrap(RequestTracing::new())
            .service(handlers::songs)
    })
    .bind("0.0.0.0:8081")?
    .run()
    .await
}
```


Instrumentation libraries - what you get

Traces >  songs-lib > GET /songs/{title}/{artist} > trace_id 8f8b1e07731500ca8dafcd0a7eb72472

p49

🕒 15.8ms

GET 200 OK

Aug 31 13:48:44.560 (just now)

Trace: Flame Graph Waterfall Span List 1 Map

🔍 Filter spans by any attribute

Color by Service ▼



0

5 ms

10 ms

15 ms

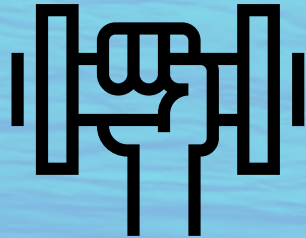


GET /songs/{title}/{artist}

200

15.8ms

Manual instrumentation



Manual instrumentation - how to

```
app.get('/songs/:title/:artist', async (req, res) => {
  const title = req.params.title
  const artist = req.params.artist
  const host = req.headers['host']

  // Extract trace context from incoming headers
  const activeContext = opentelemetry.propagation.extract(opentelemetry.context.active(), req.headers)

  const span = tracer.startSpan('GET /songs/:title/:artist', {
    kind: 1, // server
    attributes: {
      [ATTR_HTTP_REQUEST_METHOD]: HTTP_REQUEST_METHOD_VALUE_GET,
      [ATTR_HTTP_ROUTE]: '/songs/:title/:artist',
      [ATTR_SERVER_ADDRESS]: host,
      [ATTR_SERVER_PORT]: host.split(':')[1],
      [ATTR_URL_FULL]: `http://${host}/songs/${title}/${artist}`,
      [ATTR_URL_PATH]: `/songs/${title}/${artist}`,
      [ATTR_URL_SCHEME]: req.query.scheme,
      [ATTR_USER_AGENT_ORIGINAL]: req.headers['user-agent'],
    }
  }, activeContext)

  // [...]

  span.end()
}
```

Instrumentation libraries - what you get

Traces >  songs-manual > GET /songs/:title/:artist > trace_id db65ea8916610f6ed9a42c291a3ea3ab

 5.68ms | GET http://localhost:3000/songs/smells%20like%20tee... 200 OK Aug 31 13:59:12.590 (just now)

Trace: Flame Graph Waterfall Span List 1 Map

 Filter spans by any attribute

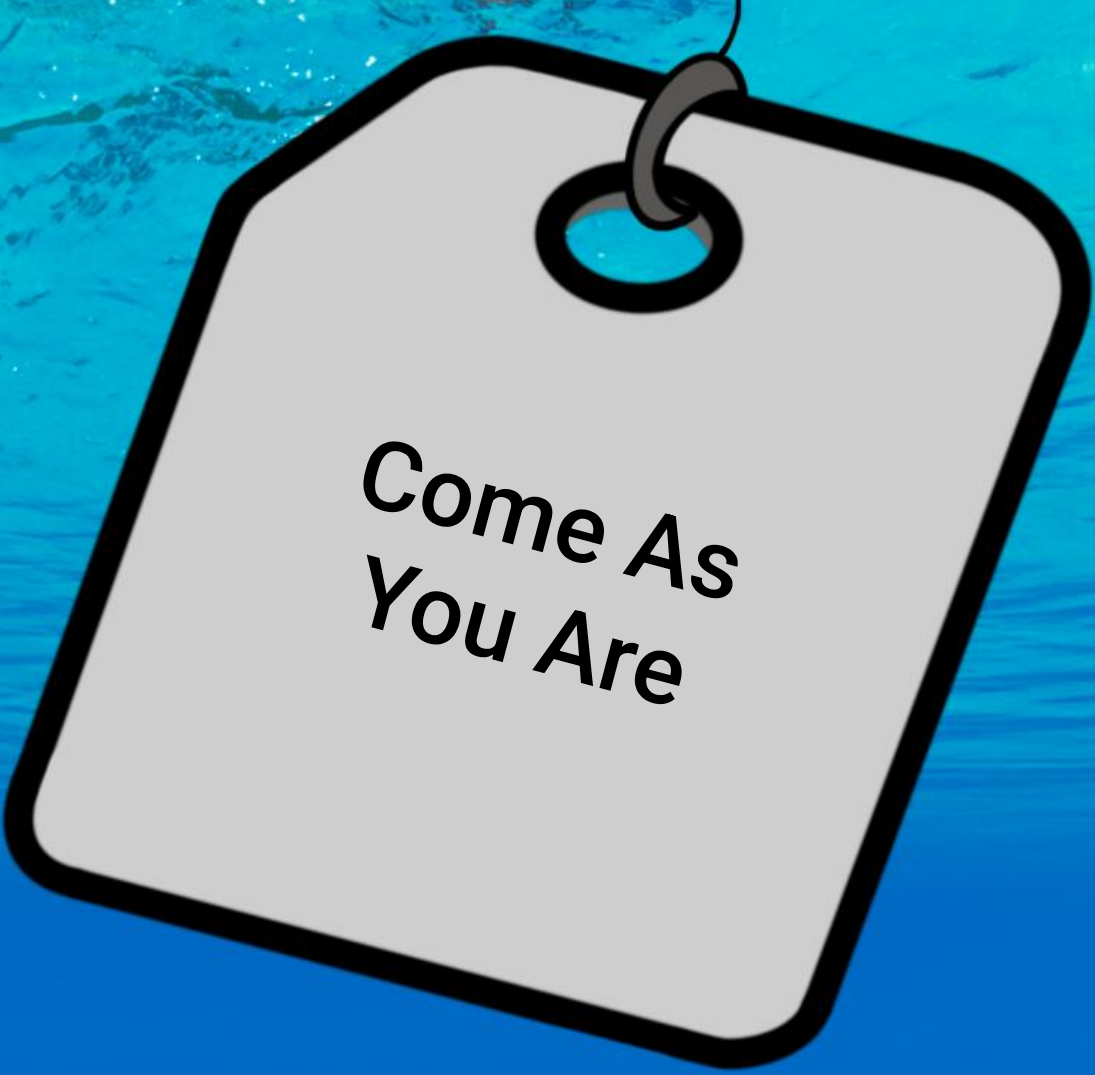
Color by Service ▼



0

5 ms

 GET /songs/:title/:artist	200		5.68ms
---	-----	---	--------

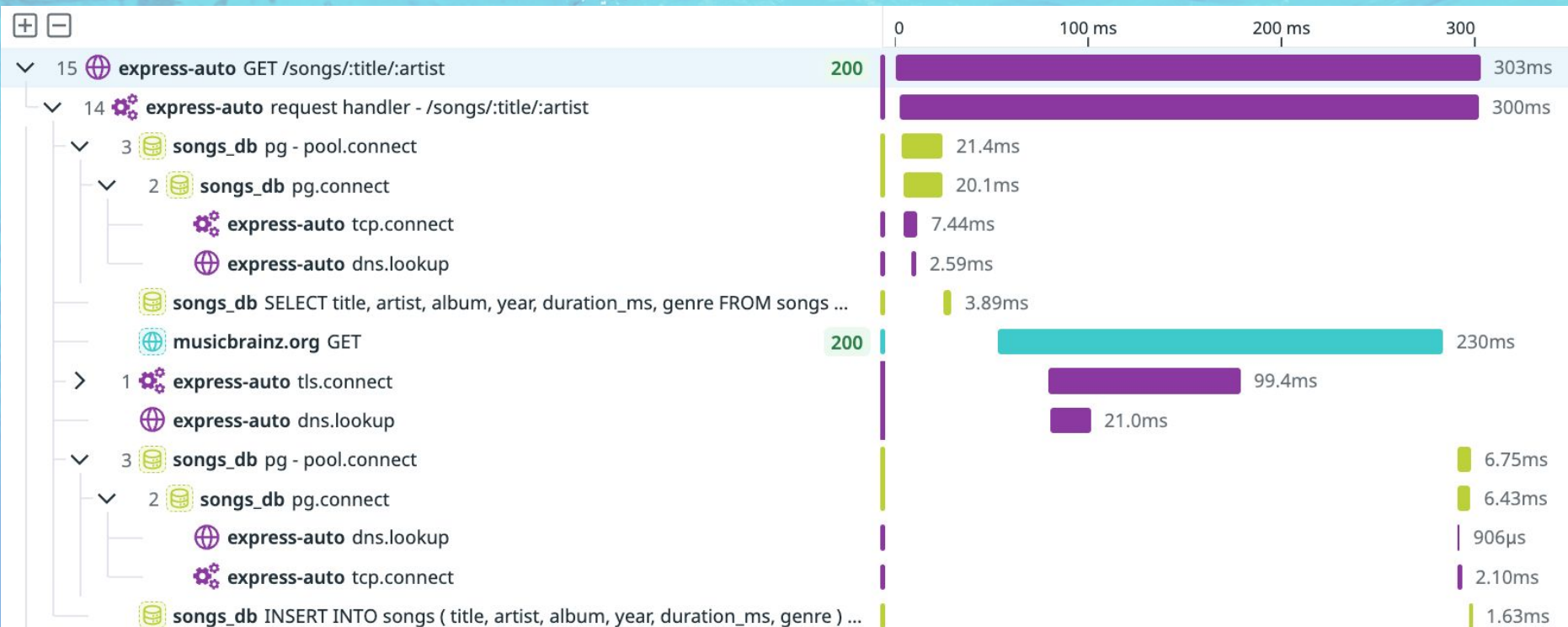


**Come As
You Are**

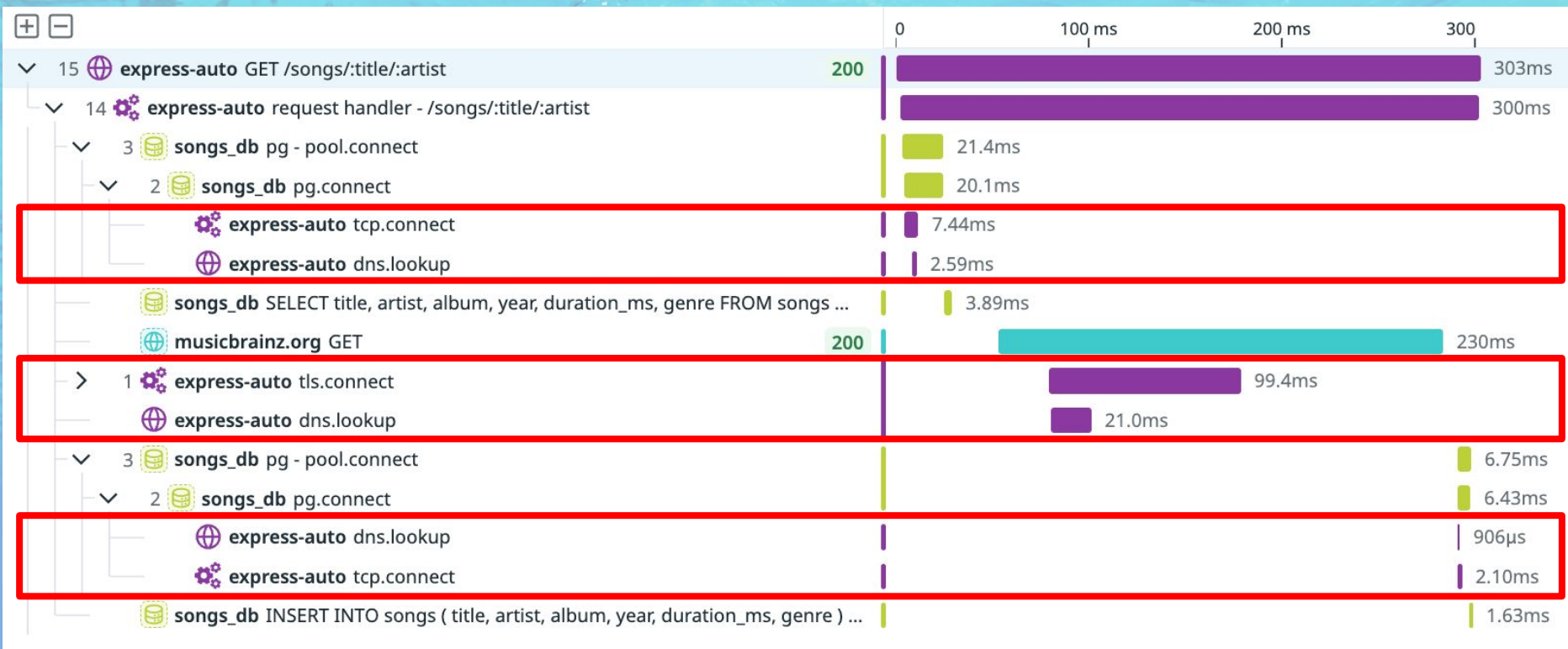


~~Come As
You Are~~

Come As You Are



Come As You Are



Come As You Are

```
const {
  getNodeAutoInstrumentations,
} = require('@opentelemetry/auto-instrumentations-node');

const sdk = new NodeSDK({
  traceExporter: new OTLPTraceExporter(),
  instrumentations: [
    getNodeAutoInstrumentations({
      '@opentelemetry/instrumentation-net': { enabled: false },
      '@opentelemetry/instrumentation-dns': { enabled: false },
    })
  ],
});
```



**Something In
The Way**

Something In The Way

```
processors:  
  filter/ottl:  
    error_mode: ignore  
  traces:  
    span:  
      - IsMatch(instrumentation_scope.name, "@opentelemetry/instrumentation-net")  
      - IsMatch(instrumentation_scope.name, "@opentelemetry/instrumentation-dns")
```

Something In The Way

Traces >  express-auto > GET /songs/:title/:artist > trace_id 5698097b85c4212405ebbdadffaf5aff

🕒 296ms | GET http://localhost:3000/songs/smells%20like%20tee... 200 OK Sep 01 12:12:51.655 (just now)

Trace: Flame Graph Waterfall Span List 9 Map

🔍 Filter spans by any attribute

Color by Service ▼



Something In The Way

Traces >  express-auto > GET /songs/:title/:artist > trace_id 9d63f6104dbc2ef61511df8b13898c96

p67 ⌚ 218ms | GET http://localhost:3000/songs/smells%20like%20tee... 200 OK Sep 01 12:25:50.541 (13m ago)

Trace: Flame Graph Waterfall Span List 8 Map

🔍 Filter spans by any attribute

Color by Service ▼



0 50 ms 100 ms 150 ms 200 ms





Milk It*

Milk It

```
pub async fn find_by_title_and_artist(title: &str, artist: &str) -> Result<String, String> {  
    get_active_span(|span| {  
        span.set_attributes(vec![  
            opentelemetry::KeyValue::new("media.song.name", title.to_string()),  
            opentelemetry::KeyValue::new("media.artist.name", artist.to_string()),  
            opentelemetry::KeyValue::new(USER_ID, "12345".to_string()),  
        ]);  
    });  
    // [...]  
}
```

Milk It

```
async fn get_song_from_db(title: &str, artist: &str) -> Result<Vec<Row>, String> {  
    let tracer = opentelemetry::global::tracer("music_service");  
    let parent_cx = Context::current();  
  
    let mut span = tracer  
        .span_builder("SELECT songs_db.songs")  
        .with_kind(SpanKind::Client)  
        .with_attributes(vec![  
            opentelemetry::KeyValue::new(DB_SYSTEM_NAME, "postgresql"),  
            opentelemetry::KeyValue::new(  
                DB_QUERY_TEXT,  
                "SELECT * FROM songs WHERE title ILIKE $1 AND artist ILIKE $2",  
            ),  
            opentelemetry::KeyValue::new(DB_OPERATION_NAME, "SELECT"),  
        ])  
        .start_with_context(&tracer, &parent_cx);  
  
    // [...]  
}
```


Milk It

Traces >  songs-lib > GET /songs/{title}/{artist} > trace_id 6e4e082fc8b15f5c48c78454df100174

 7.34ms | GET 200 OK Sep 01 15:32:25.554 (just now)

Trace: Flame Graph Waterfall Span List 2 Map

 Filter spans by any attribute

Color by Service ▼



▼ 1  GET /songs/{title}/{artist}

 SELECT songs_db.songs

200

0

5 ms

7.34ms

7.31ms



Stay Away

Stay Away

Traces >  songs-lib > GET /songs/{title}/{artist} > trace_id 42a3a7ed29b8fdabc3168aea51d9f89c

p67 ⌚ 168ms | GET 200 OK Sep 01 16:07:33.210 (just now)

Trace: Flame Graph Waterfall Span List 8 Map

🔍 Filter spans by any attribute

Color by Service ▼



7  songs-lib GET /songs/{title}/{artist}

200



 songs-lib SELECT songs_db.songs

16.3ms

 songs-lib connect database

14.9ms

 musicbrainz.org GET

200

142ms

 songs-lib extract album and year

7.46μs

 songs-lib extract genre

875ns

 songs-lib INSERT songs_db.songs

8.58ms

 songs-lib connect database

7.39ms

Stay Away

Traces >  songs-lib > GET /songs/{title}/{artist} > trace_id 96549489fee6f41ef9334a21b796c87e

 293ms | GET 200 OK Sep 01 16:10:03.505 (just now)

Trace: Flame Graph Waterfall Span List 4 Map

 Filter spans by any attribute

Color by Service 





Drain You

Drain You

```
http.request_method
```

```
http.request
```

```
http.method
```

Drain You



Drain You

```
http.request.method
```

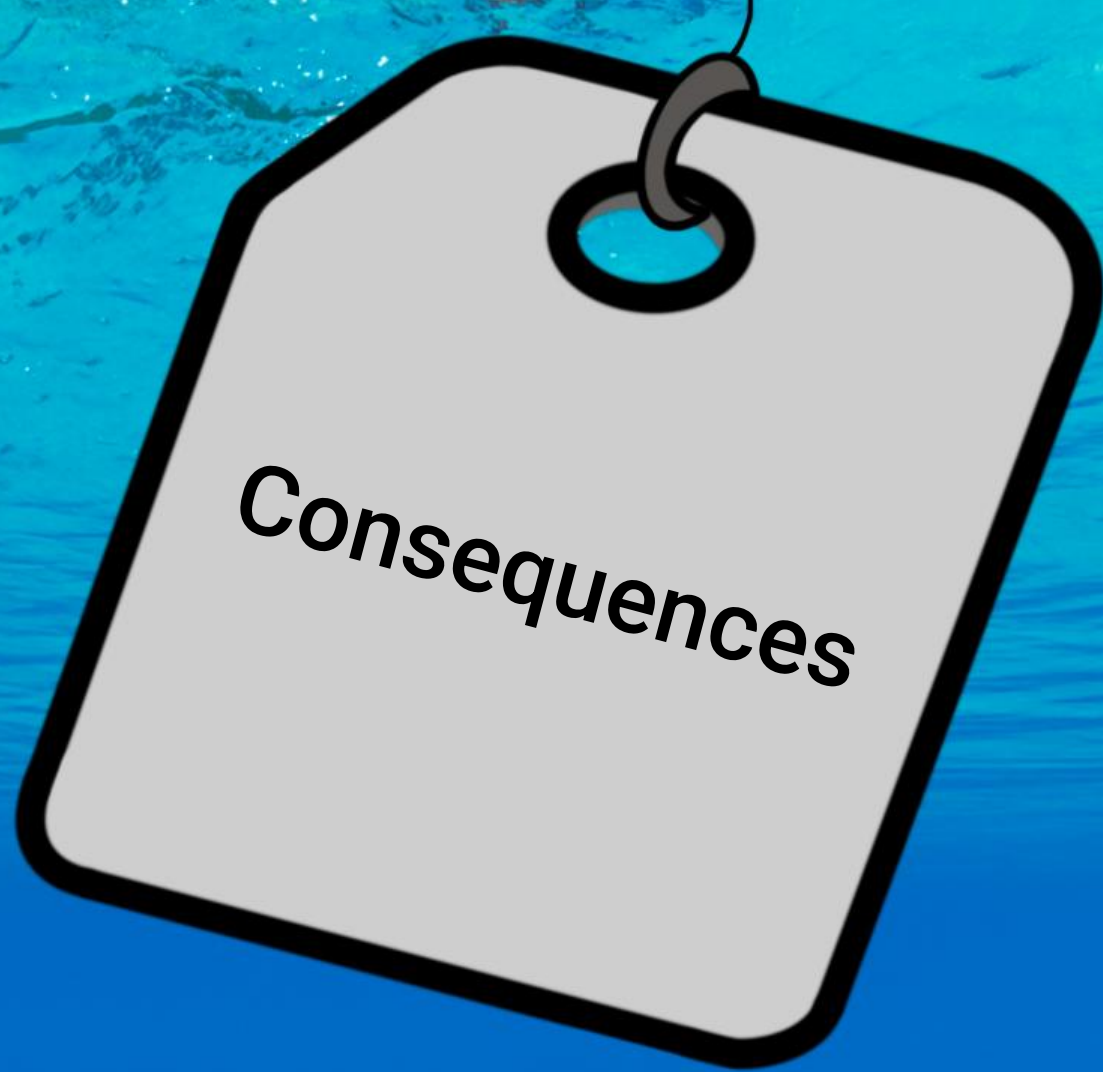
<https://opentelemetry.io/docs/specs/semconv/http/http-spans/>

Drain You

```
deployment.environment.name: "production"
```

```
deployment.environment.name: "prod"
```

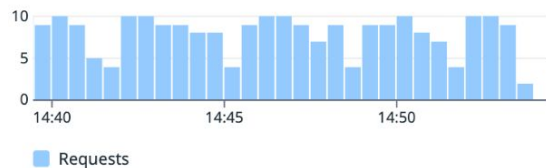
```
deployment.environment.name: "prd"
```



Consequences

▼ Requests, Errors, and Latency

Requests 231 total (0.3 req/s)



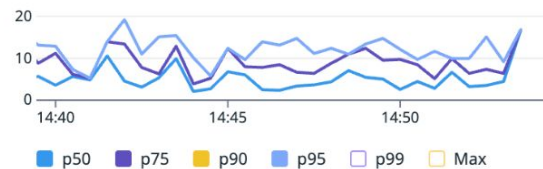
Errors

Analyze →



Latency

Analyze →



Consequences

env:production

||

env:prod

||

env:prd

A white, rectangular tag with rounded corners and a thick black border is shown. A small grey ring is attached to the top edge of the tag. The tag is tilted slightly to the right. The background is a vibrant blue with a wavy, textured pattern, resembling water or a stylized background. The word "Endless" is written in a bold, black, sans-serif font, centered on the tag and tilted to match its angle.

Endless

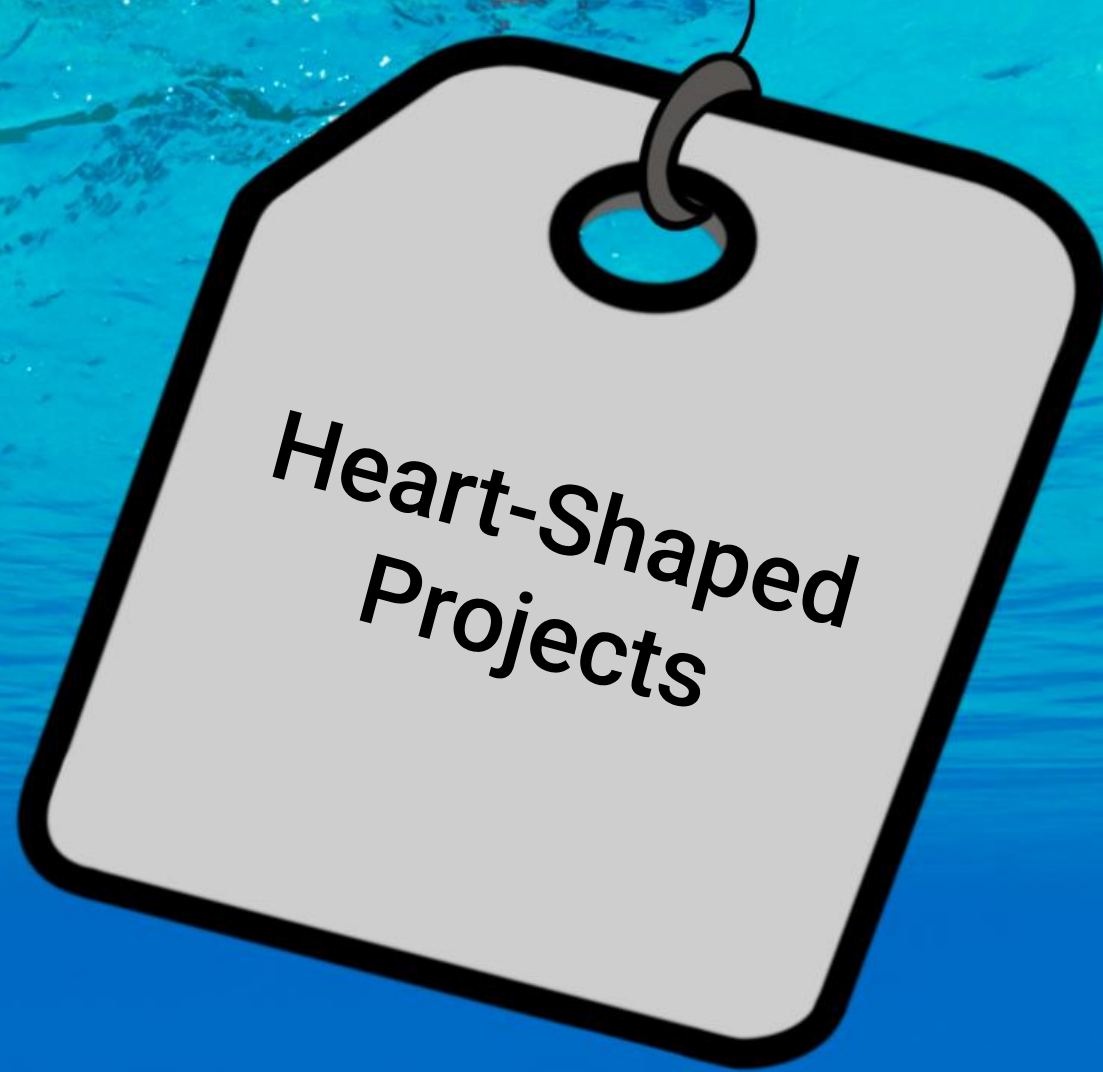
Endless

Instrument



Prune/Add

Validate



**Heart-Shaped
Projects**

Heart-Shaped Projects - Instrumentation Score

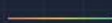
Instrumentation Score

A standardized, vendor-neutral metric for assessing
OpenTelemetry instrumentation quality.

From 0 to 100. Objective. Actionable. Community-driven.

Calculated using weighted scoring based on rule criticality against OpenTelemetry semantic conventions and community best practices.

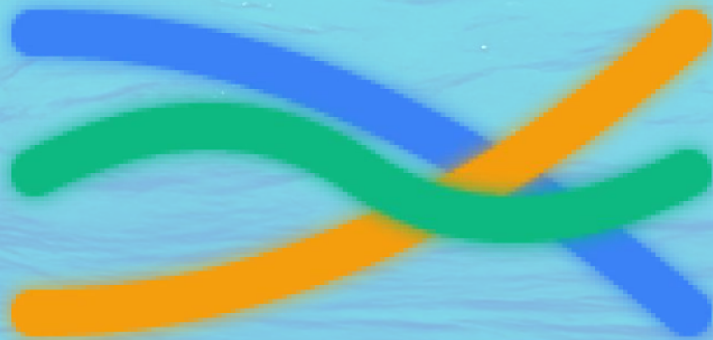
 **0** Poor



 **100** Excellent

<https://score.olly.garden/>

Heart-Shaped Projects - OTel Weaver



OpenTelemetry Weaver

<https://github.com/open-telemetry/weaver>

Heart-Shaped Projects - OTel Weaver

Attribute	Type	Description	Examples	Stability
<code>media.album.name</code>	string	The name of the album containing the song.	<code>Nevermind</code> ; <code>In Utero</code> ; <code>Bleach</code> ; <code>Incesticide</code>	stable
<code>media.artist.name</code>	string	The name of the artist performing the song.	<code>Nirvana</code> ; <code>Foo Fighters</code> ; <code>Linkin Park</code> ; <code>Slipknot</code>	stable
<code>media.song</code>	string	The name/title of the song being queried. [1]	<code>Smells Like Teen Spirit</code> ; <code>Come As You Are</code> ; <code>Lithium</code> ; <code>Something In The Way</code>	deprecated Replaced by <code>media.song.name</code> .
<code>media.song.duration_ms</code>	int	The duration of the song in milliseconds.	<code>301000</code> ; <code>355000</code> ; <code>183000</code> ; <code>391000</code>	stable
<code>media.song.genre</code>	string	The musical genre of the song.	<code>Rock</code> ; <code>Punk</code> ; <code>Grunge</code> ; <code>Classical</code> ; <code>Hip Hop</code>	stable
<code>media.song.name</code>	string	The name/title of the song being queried.	<code>Smells Like Teen Spirit</code> ; <code>Come As You Are</code> ; <code>Lithium</code> ; <code>Something In The Way</code>	stable
<code>media.song.year</code>	int	The release year of the song.	<code>1991</code> ; <code>1975</code> ; <code>1971</code> ; <code>1976</code>	stable
<code>user.id</code>	string	Unique identifier of the user.	<code>S-1-5-21-202424912787-2692429404-2351956786-1000</code>	development

[1] `media.song`: This attribute is deprecated and will be removed in future versions. Use `media.song.name` for new implementations.

Heart-Shaped Projects - OTel Weaver

```
/**
 * <p>The name of the album containing the song.</p>
 * <h2>Examples</h2>
 * <ul>
 *   <li><c>"Nevermind"</c></li>
 *   <li><c>"In Utero"</c></li>
 *   <li><c>"Bleach"</c></li>
 *   <li><c>"Incesticide"</c></li>
 * </ul>
 */
public static final String MEDIA_ALBUM_NAME = "media.album.name";

/**
 * <p>The name/title of the song being queried.</p>
 * <h2>Examples</h2>
 * <ul>
 *   <li><c>"Smells Like Teen Spirit"</c></li>
 *   <li><c>"Come As You Are"</c></li>
 *   <li><c>"Lithium"</c></li>
 *   <li><c>"Something In The Way"</c></li>
 * </ul>
 */
public static final String MEDIA_SONG_NAME = "media.song.name";
```


Heart-Shaped Projects - OTel Weaver

Resource

```
telemetry.sdk.language = python
telemetry.sdk.name = opentelemetry
telemetry.sdk.version = 1.35.0
service.version = 0.0.1
service.name = manual-python
```

Span GET /songs/{title}/{artist} `server`

`media.song` = smells like teen spirit

- deprecated: `renamed` - Replaced by ``media.song.name``.

`media.artist` = nirvana

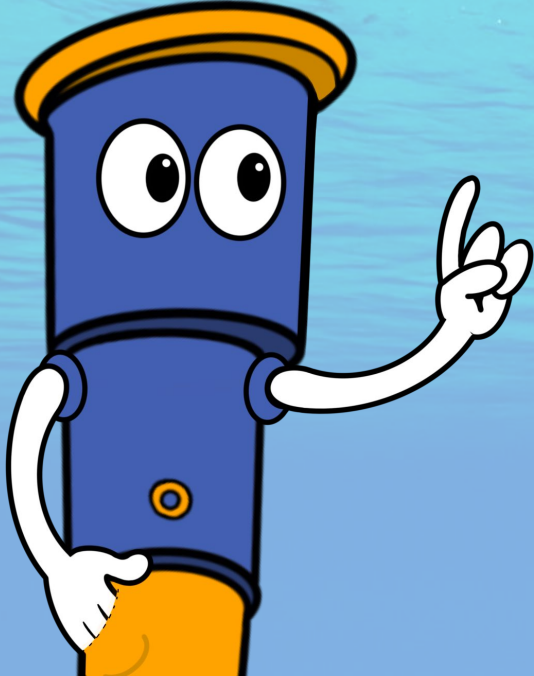
- missing_attribute: `media.artist` - Does not exist in the registry
- extends_namespace: `media` - Extends existing namespace



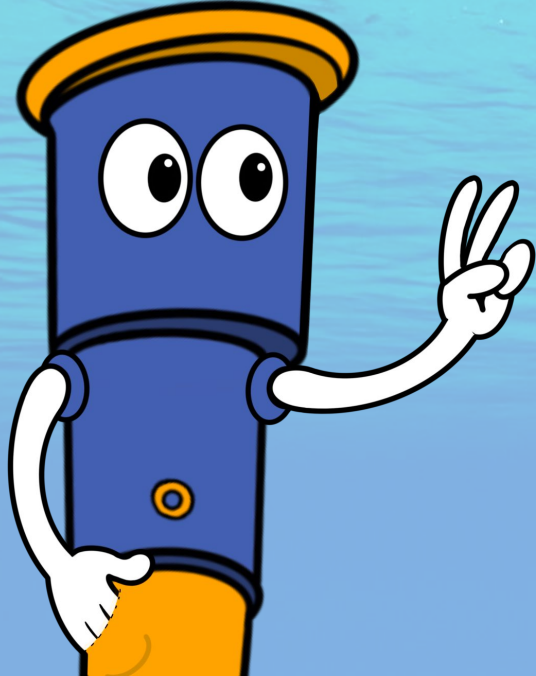
No Apologies

No Apologies

Use logs only where you have to



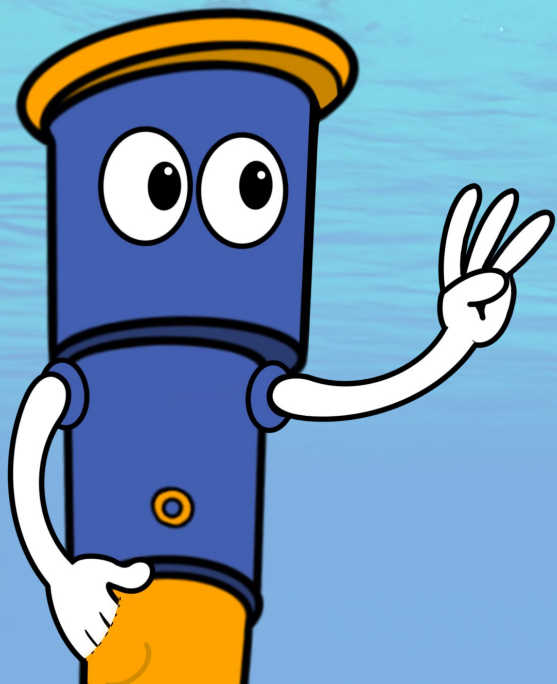
No Apologies



Enrich existing spans
Add spans where you don't have visibility

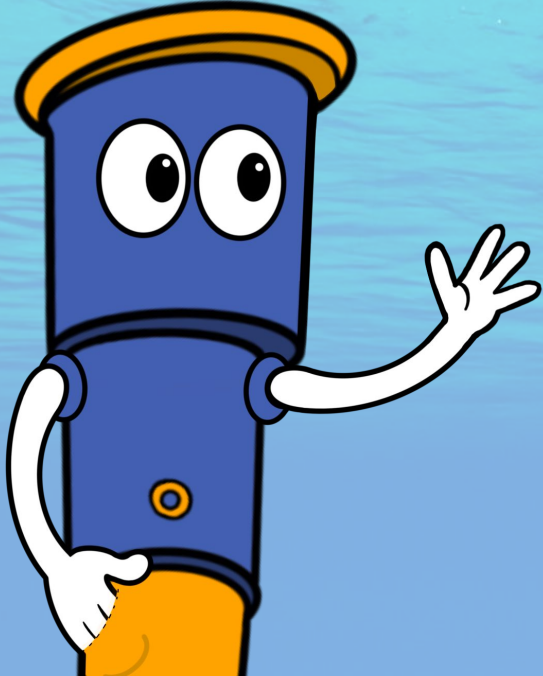
No Apologies

Do not over-instrument

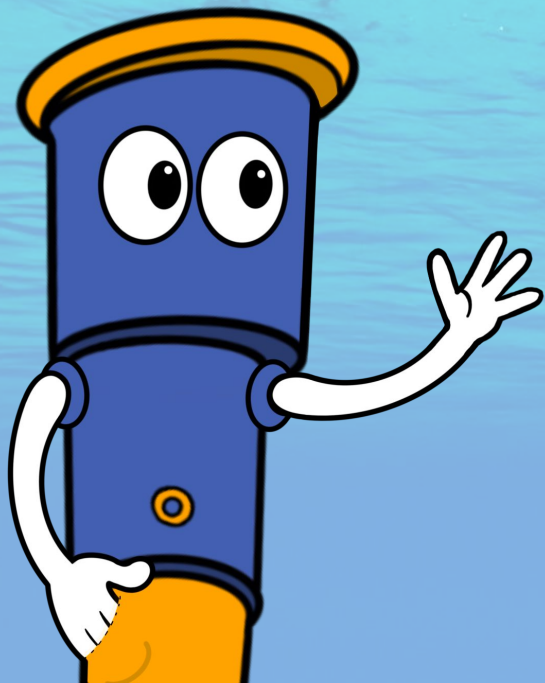


No Apologies

Revisit, validate and evolve as you go



No Apologies



Use logs only where you have to

Enrich existing spans
Add spans where you don't have visibility

Do not over-instrument

Revisit, validate and evolve as you go

Thank you!



Come grab
stickers!



Juliano Costa

Developer Advocate at Datadog



jcosta.dev